# Processes, Threads, and Memory

## Video 1 · Processes and Their Memory

How Computers Work · Section 6

In this video, you will learn:

- What a process is (vs a program)
- How a process is laid out in memory
- The stack and the heap

# Program vs process

- A **program** is a file on disk: instructions, doing nothing. (The recipe.)
- A **process** is a program **running**: loaded into memory, CPU executing it. (Cooking the dish.)

One program → many processes (two windows of one app).

Starting one, the OS: loads it into RAM, gives it its own memory, assigns a **PID**, and runs it. Each process has a **parent** (**PPID**), what started it. *What started what* reveals attacks.

# A process's memory layout

The OS gives a process a strip of memory, organized into regions:

| Region | Holds | |
|---|---|---|
| **Code (text)** | The instructions | (reverse engineers read this) |
| **Data** | Globals, constants | |
| **Heap** | Requested-while-running memory | grows **up** ↑ |
| **Stack** | Function data | grows **down** ↓ |

Heap and stack grow **toward each other**. Attacks live in this map.

# The stack and the heap

**Stack:** orderly, automatic, function data. Each function call adds a **stack frame:** local variables + the **return address** (where to continue after).

> A **buffer overflow** overwrites the **return address**, so the CPU jumps to the attacker's code. (Lesson 11, Binary Exploitation.)

**Heap:** flexible, longer-lived memory the program requests and must give back. Mistakes → **use-after-free**, leaks.

**Stack = orderly/automatic; heap = flexible/manual.**

# Recap: processes and memory

- **Program** = file on disk; **process** = it running in memory; **PID/PPID** track it and its parent.
- Memory: code and data at the bottom; **heap grows up**, **stack grows down**.
- The **stack** holds the **return address**, the classic exploit target.

**Next video:** threads, isolation, and seeing real processes.

# Processes, Threads, and Memory

## Video 2 · Threads, Isolation, and Seeing Processes

How Computers Work · Section 6

In this video, you will learn:

- Threads, and how they differ from processes
- Virtual memory and isolation
- Looking at running processes with `ps`

# Threads

A **thread** is a single stream of execution inside a process. A process has at least one, and can make more to do several things at once.

- **Processes are isolated:** each has its own memory; one can't see another's.
- **Threads share** their process's memory, fast, but dangerous.

Shared memory + unpredictable timing → **race conditions**: bugs (and vulnerabilities) where the outcome depends on exact thread timing. (Web security, exploitation.)

# Virtual memory and isolation

What stops one process reading another's memory? **Virtual memory.**

- Each process gets its **own private view** of memory (a **virtual address space**), as if alone on the machine.
- The CPU/OS translate those virtual addresses to **different** physical RAM per process.

So address `0x1000` in two processes points to **different** physical memory. One process **can't name**, let alone touch, another's data.

**This isolation keeps your banking app safe from a malicious app beside it.**

# Seeing real processes: `ps`

```
$ ps -o pid,ppid,%cpu,%mem,comm
  PID  PPID %CPU %MEM COMM
 1521     1   0.3  1.8 WindowServer
 2983  2970   1.2  4.1 Google Chrome
 3001  2983   0.0  2.2 Google Chrome Helper
 4102  3550   0.1  0.4 zsh
```

- **PID** = unique ID; **PPID** = parent. Helper (3001) → parent Chrome (2983).
- Defenders scan for the abnormal: a **document app whose child is a shell**, a strange name, odd CPU.

# Recap: threads and isolation

- **Threads** share a process's memory (fast, but → race conditions); **processes** are isolated.
- **Virtual memory** gives each process a private address space, the isolation that keeps them apart.
- A computer is a set of **boundaries** (process/process, user/kernel, read/write/execute); attacks break them, defenses keep them.
- `ps` shows real processes, finding malware starts with reading this.

**Next lesson:** where programs and data rest when not running, files and file systems.